THREE-TIER APPLICATION DEPLOYMENT ON AWS LAMBDA

DYNAMODB

1. go to dynamodb
2. select a table
    1. 1.table name - Students
    2. 2.partition key - studentId
    3. 3.type - string
    4. 4.choose default settings
    5. 5.create table

Open the Students table and confirm that its status is Active and its partition key is studentId (String).

CREATE A ROLE IN IAM

1. create a role
2. select aws service
3. use case - lambda
4. permission policies - DynamoDBFullAccess/AdministratorAccess
5. role name - lambda-role
6. create a role

BACKEND ON AWS LAMBDA

1. create the lambda function
    1. click on create function
    2. select author from scratch
    3. function name - StudentManagementAPI
    4. Runtime - Python 3.13
    5. execution role - lambda-role
    6. create a function

Giving Lambda Granular Access to DynamoDB

1. click on lambda function
2. select configuration - permissions
3. Click the execution role link to open IAM.
4. Choose Add permissions - Create inline policy.
5. Select the JSON editor
6. paste the policy, replacing YOUR_ACCOUNT_ID with your AWS account ID.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
```

```
      "dynamodb:PutItem",
      "dynamodb:Scan"
    ],
    "Resource": "arn:aws:dynamodb:ap-south-1:YOUR_ACCOUNT_ID:table/Students"
  }
 ]
}
```

7. policy name – StudentsTableAccess
8. create the policy

This permits Lambda to save and list student records in this table only.

CONFIGURE THE ENVIRONMENT VARIABLE

1. select configuration - Environment variables
2. click edit - Add environment variable
3. Enter
   key - STUDENTS_TABLE
   value - Students
4. save changes

ADDING THE PYTHON CODE

1.add the code in lambda_function.py

```
import json
import os
import uuid
import boto3
from decimal import Decimal

dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table(os.environ["STUDENTS_TABLE"])


def convert(value):
  if isinstance(value, Decimal):
    return float(value)
  raise TypeError(f"Cannot serialize {type(value).__name__}")


def reply(status, data):
  return {
    "statusCode": status,
    "headers": {
      "Content-Type": "application/json"
    },
    "body": json.dumps(data, default=convert)
  }


def lambda_handler(event, context):
  method = (
    event.get("requestContext", {})
```

```
    .get("http", {})
    .get("method", "GET")
)

path = event.get("rawPath", "/").rstrip("/") or "/"

# Remove the API stage prefix, such as /default.
stage = event.get("requestContext", {}).get("stage", "")

if stage and stage != "$default":
    prefix = f"/{stage}"
    if path == prefix:
        path = "/"
    elif path.startswith(prefix + "/"):
        path = path[len(prefix):]

# POST /students: create a student record.
if method == "POST" and path == "/students":
    try:
        data = json.loads(event.get("body") or "{}")
    except (json.JSONDecodeError, TypeError):
        return reply(400, {"message": "Invalid JSON"})

    if not isinstance(data, dict):
        return reply(400, {
            "message": "JSON body must be an object"
        })

    required = [
        "name", "rollNumber", "mobile",
        "percentage", "education", "branch"
    ]

    if any(
        data.get(key) is None or not str(data.get(key)).strip()
        for key in required
    ):
        return reply(400, {
            "message": "All student fields are required"
        })

    try:
        percentage = Decimal(str(data["percentage"]))
    except Exception:
        return reply(400, {
            "message": "Percentage must be a number"
        })

    if (
        not percentage.is_finite()
        or not Decimal("0") <= percentage <= Decimal("100")
    ):
        return reply(400, {
            "message": "Percentage must be between 0 and 100"
        })

    student = {
        "studentId": str(uuid.uuid4()),
        "name": str(data["name"]).strip(),
        "rollNumber": str(data["rollNumber"]).strip(),
        "mobile": str(data["mobile"]).strip(),
        "percentage": percentage,
        "education": str(data["education"]).strip(),
        "branch": str(data["branch"]).strip()
    }
```

```
    try:
        table.put_item(Item=student)
    except Exception:
        return reply(500, {
            "message": "Could not save student. Check Lambda permissions and DynamoDB."
        })

    return reply(201, {
        "message": "Student saved successfully",
        "student": student
    })

# GET /students: list student records.
if method == "GET" and path == "/students":
    try:
        result = table.scan()
        students = result.get("Items", [])

        while "LastEvaluatedKey" in result:
            result = table.scan(
                ExclusiveStartKey=result["LastEvaluatedKey"]
            )
            students.extend(result.get("Items", []))

        return reply(200, {
            "count": len(students),
            "students": students
        })
    except Exception:
        return reply(500, {
            "message": "Could not retrieve students. Check Lambda permissions and DynamoDB."
        })

return reply(404, {
    "message": "Route not found",
    "method": method,
    "path": path
})
```

2.click on deploy and wait for the success message

CREATE API GATEWAY AS A TRIGGER FOR LAMBDA

API Gateway gives your backend public HTTP endpoints that the frontend can call.

Create API Gateway

1. click on the lambda function
2. click on trigger
3. click on add trigger
4. select API gateway
5. choose create a new API
6. select HTTP API
7. security - OPEN
8. create the API

Open API Gateway and configure routes

1. open the API gateway console
2. click on API - StudentManagementAPI-API

3. Add the routes
    1. Route1 - GET
    2. click create
    3. select method GET
    4. path - /students
    5. create the route
    6. Select it and click Attach integration if no integration is attached.
    7. choose lambda - StudentManagementAPI - Save

4. Route2 - POST
    1. click create
    2. select method POST
    3. path - /students
    4. create the route
    5. Select it and attach the same lambda integration
    6. choose lambda - StudentManagementAPI - Save

Configure the stage

1. Click Stages.
2. Select the default stage created by the trigger.
3. Confirm Automatic deployment is enabled.
4. If it is disabled, deploy the API to default.

Configure CORS for the frontend

1. Allowed origins - *
2. Allowed methods - GET, POST, OPTIONS
3. Allowed headers - content-type
4. Allow credentials - Disabled
5. save the configuration. Deploy manually if automatic deployment is disabled.

Test GET

1. Open your actual Invoke URL followed by /students
2. click on API gateway - APIs - StudentManagementAPI-API
   https://9ut6kaxrra.execute-api.ap-south-1.amazonaws.com/default/students

Expected Result:

```
{
  "count": 0,
  "students": []
}
```

FRONTEND ON S3

1. create a index.html file

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Student Management</title>
 <style>
  :root {
   color-scheme: light;
   --ink: #172033;
   --muted: #687386;
   --line: #e3e8f0;
   --panel: #ffffff;
   --bg: #f4f7fb;
   --brand: #3157d5;
   --brand-dark: #2343b4;
   --green: #16794b;
   --red: #b42318;
   --shadow: 0 12px 35px rgba(22, 34, 58, .07);
  }
  * { box-sizing: border-box; }
  body {
   margin: 0;
   background: var(--bg);
   color: var(--ink);
   font: 15px/1.5 system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  }
  .topbar {
   color: white;
   background: linear-gradient(120deg, #172b66, #3157d5);
   padding: 28px 20px;
  }
  .topbar-inner, main { width: min(1120px, calc(100% - 32px)); margin: 0 auto; }
  .eyebrow { margin: 0 0 5px; font-size: 12px; letter-spacing: .14em; text-transform: uppercase; opacity: .8; font-weight: 700; }
  h1 { margin: 0; font-size: clamp(25px, 4vw, 34px); letter-spacing: -.03em; }
  .subtitle { margin: 7px 0 0; color: #e0e7ff; }
  main { padding: 26px 0 44px; }
  .stats { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 14px; margin-bottom: 18px; }
  .stat, .panel { background: var(--panel); border: 1px solid var(--line); border-radius: 16px; box-shadow: var(--shadow); }
  .stat { padding: 16px 18px; }
  .stat-label { color: var(--muted); font-size: 13px; }
  .stat-value { font-size: 24px; font-weight: 750; margin-top: 2px; }
  .layout { display: grid; grid-template-columns: minmax(300px, .85fr) minmax(0, 1.5fr); gap: 18px; align-items: start; }
  .panel { padding: 22px; min-width: 0; }
  .panel-head { display: flex; align-items: flex-start; justify-content: space-between; gap: 12px; margin-bottom: 18px; }
  h2 { font-size: 19px; margin: 0; letter-spacing: -.02em; }
  .panel-description { color: var(--muted); margin: 4px 0 0; font-size: 13px; }
  .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
  .field { display: flex; flex-direction: column; gap: 6px; min-width: 0; }
  .field.full { grid-column: 1 / -1; }
  label { font-weight: 650; font-size: 13px; }
  input, select {
   width: 100%; min-width: 0; padding: 11px 12px; border: 1px solid #cfd7e4;
   border-radius: 9px; background: white; color: var(--ink); font: inherit; outline: none;
  }
  input:focus, select:focus { border-color: var(--brand); box-shadow: 0 0 0 3px rgba(49, 87, 213, .12); }
  .actions { display: flex; gap: 10px; align-items: center; margin-top: 18px; }
  button { border: 0; border-radius: 9px; padding: 11px 15px; cursor: pointer; font: inherit; font-weight: 700; }
  button:disabled { opacity: .6; cursor: wait; }
  .primary { background: var(--brand); color: white; flex: 1; }
  .primary:hover:not(:disabled) { background: var(--brand-dark); }
  .secondary { background: #eef2ff; color: #2948b8; }
  .notice { display: none; padding: 11px 12px; margin: 0 0 15px; border-radius: 9px; font-size: 13px; overflow-wrap: anywhere; }
  .notice.show { display: block; }
  .notice.success { background: #e9f8ef; color: var(--green); }
  .notice.error { background: #fff0ef; color: var(--red); }
```

```
   .table-wrap { overflow-x: auto; border: 1px solid var(--line); border-radius: 11px; }
   table { width: 100%; border-collapse: collapse; min-width: 720px; background: white; }
   th, td { padding: 12px 13px; text-align: left; border-bottom: 1px solid var(--line); white-space: nowrap; }
   th { background: #f8faff; color: #536078; font-size: 11px; text-transform: uppercase; letter-spacing: .06em; }
   tbody tr:last-child td { border-bottom: 0; }
   tbody tr:hover { background: #fafbff; }
   .empty { text-align: center; padding: 32px 16px; color: var(--muted); white-space: normal; }
   .table-footer { display: flex; justify-content: space-between; align-items: center; gap: 12px; margin-top: 13px; color: var(--muted); font-
size: 13px; }
   .pill { background: #eef2ff; color: #2948b8; border-radius: 99px; padding: 4px 9px; font-size: 12px; font-weight: 700; }
   .footer-note { margin: 18px 2px 0; color: var(--muted); font-size: 12px; }
   @media (max-width: 850px) { .layout { grid-template-columns: 1fr; } }
   @media (max-width: 540px) {
    .stats { grid-template-columns: 1fr; gap: 9px; }
    .form-grid { grid-template-columns: 1fr; }
    .field.full { grid-column: auto; }
    .panel { padding: 17px; }
    .topbar { padding: 23px 0; }
    .table-footer { align-items: flex-start; flex-direction: column; }
   }
  </style>
</head>
<body>
  <header class="topbar">
   <div class="topbar-inner">
    <p class="eyebrow">Student records · AWS serverless demo</p>
    <h1>Student Management</h1>
    <p class="subtitle">Register students and view records saved in the cloud.</p>
   </div>
  </header>

  <main>
   <section class="stats" aria-label="Student summary">
    <div class="stat"><div class="stat-label">Total students</div><div class="stat-value" id="totalStudents">—</div></div>
    <div class="stat"><div class="stat-label">API status</div><div class="stat-value" id="apiStatus" style="font-
size:18px">Connecting…</div></div>
    <div class="stat"><div class="stat-label">Storage</div><div class="stat-value" style="font-size:18px">Amazon DynamoDB</div></div>
   </section>

   <div class="layout">
    <section class="panel">
     <div class="panel-head">
      <div><h2>Register a student</h2><p class="panel-description">All fields are required.</p></div>
     </div>
     <div id="formNotice" class="notice" role="status" aria-live="polite"></div>
     <form id="studentForm">
      <div class="form-grid">
       <div class="field full">
        <label for="name">Student name</label>
        <input id="name" name="name" autocomplete="name" maxlength="100" required placeholder="e.g. Rahul Sharma" />
       </div>
       <div class="field">
        <label for="rollNumber">Roll number</label>
        <input id="rollNumber" name="rollNumber" maxlength="40" required placeholder="e.g. 101" />
       </div>
       <div class="field">
        <label for="mobile">Mobile number</label>
        <input id="mobile" name="mobile" type="tel" inputmode="numeric" pattern="[0-9+() -]{7,20}" maxlength="20" required
placeholder="e.g. 9876543210" />
       </div>
       <div class="field">
        <label for="percentage">Percentage</label>
        <input id="percentage" name="percentage" type="number" min="0" max="100" step="0.01" required placeholder="e.g. 85.5" />
       </div>
       <div class="field">
        <label for="education">Education</label>
```

```
          <select id="education" name="education" required>
            <option value="">Select education</option>
            <option>10th</option><option>12th</option><option>Diploma</option>
            <option>B.A.</option><option>B.Com.</option><option>B.Sc.</option>
            <option>B.E.</option><option>B.Tech</option><option>BCA</option><option>BBA</option>
            <option>M.A.</option><option>M.Com.</option><option>M.Sc.</option>
            <option>M.E.</option><option>M.Tech</option><option>MCA</option><option>MBA</option>
            <option>Other</option>
          </select>
        </div>
        <div class="field full">
          <label for="branch">Branch / specialization</label>
          <input id="branch" name="branch" maxlength="100" required placeholder="e.g. Computer Engineering" />
        </div>
      </div>
      <div class="actions">
        <button class="primary" id="saveButton" type="submit">Save student</button>
        <button class="secondary" type="reset">Clear</button>
      </div>
    </form>
  </section>

  <section class="panel">
    <div class="panel-head">
      <div><h2>Student records</h2><p class="panel-description">Records loaded from your API.</p></div>
      <button class="secondary" id="refreshButton" type="button">Refresh</button>
    </div>
    <div id="tableNotice" class="notice" role="status" aria-live="polite"></div>
    <div class="table-wrap">
      <table>
        <thead>
          <tr><th>Name</th><th>Roll no.</th><th>Mobile</th><th>Percentage</th><th>Education</th><th>Branch</th></tr>
        </thead>
        <tbody id="studentRows"><tr><td class="empty" colspan="6">Loading students…</td></tr></tbody>
      </table>
    </div>
    <div class="table-footer"><span id="recordCount">Loading records…</span><span class="pill">Live API data</span></div>
  </section>
 </div>
 <p class="footer-note">Demo application. Do not enter sensitive personal information. For production, add authentication and restrict API access.</p>
</main>

<script>
 const API_URL = "https://lfnsuyjgq1.execute-api.ap-south-1.amazonaws.com/default/students";

 const form = document.getElementById("studentForm");
 const saveButton = document.getElementById("saveButton");
 const refreshButton = document.getElementById("refreshButton");
 const rows = document.getElementById("studentRows");
 const totalStudents = document.getElementById("totalStudents");
 const apiStatus = document.getElementById("apiStatus");
 const recordCount = document.getElementById("recordCount");

 function showNotice(elementId, message, type) {
  const el = document.getElementById(elementId);
  el.textContent = message;
  el.className = `notice show ${type}`;
 }

 function clearNotice(elementId) {
  const el = document.getElementById(elementId);
  el.textContent = "";
  el.className = "notice";
 }
```

```
function makeCell(value) {
  const td = document.createElement("td");
  // textContent prevents API data from being interpreted as HTML.
  td.textContent = value === undefined || value === null ? "" : String(value);
  return td;
}

async function loadStudents() {
  refreshButton.disabled = true;
  apiStatus.textContent = "Connecting…";
  clearNotice("tableNotice");
  try {
    const response = await fetch(API_URL, { method: "GET" });
    const result = await response.json();
    if (!response.ok) throw new Error(result.message || `Request failed (${response.status})`);

    const students = Array.isArray(result.students) ? result.students : [];
    rows.replaceChildren();

    if (students.length === 0) {
      const tr = document.createElement("tr");
      const td = document.createElement("td");
      td.colSpan = 6;
      td.className = "empty";
      td.textContent = "No students yet. Add the first student using the form.";
      tr.appendChild(td);
      rows.appendChild(tr);
    } else {
      students.forEach(student => {
        const tr = document.createElement("tr");
        [student.name, student.rollNumber, student.mobile,
         student.percentage, student.education, student.branch]
          .forEach(value => tr.appendChild(makeCell(value)));
        rows.appendChild(tr);
      });
    }

    totalStudents.textContent = String(students.length);
    recordCount.textContent = `${students.length} record${students.length === 1 ? "" : "s"} found`;
    apiStatus.textContent = "Connected";
    apiStatus.style.color = "var(--green)";
  } catch (error) {
    rows.replaceChildren();
    const tr = document.createElement("tr");
    const td = document.createElement("td");
    td.colSpan = 6;
    td.className = "empty";
    td.textContent = "Could not load records. Check API Gateway CORS settings and Lambda logs.";
    tr.appendChild(td);
    rows.appendChild(tr);
    totalStudents.textContent = "—";
    recordCount.textContent = "Unable to load records";
    apiStatus.textContent = "Connection error";
    apiStatus.style.color = "var(--red)";
    showNotice("tableNotice", error.message || "API request failed. Check CORS configuration.", "error");
  } finally {
    refreshButton.disabled = false;
  }
}

form.addEventListener("submit", async event => {
  event.preventDefault();
  clearNotice("formNotice");
  saveButton.disabled = true;
  saveButton.textContent = "Saving…";
```

```
    const payload = {
      name: form.name.value.trim(),
      rollNumber: form.rollNumber.value.trim(),
      mobile: form.mobile.value.trim(),
      percentage: Number(form.percentage.value),
      education: form.education.value.trim(),
      branch: form.branch.value.trim()
    };

    try {
      const response = await fetch(API_URL, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
      });
      const result = await response.json();
      if (!response.ok) throw new Error(result.message || `Save failed (${response.status})`);

      showNotice("formNotice", "Student saved successfully.", "success");
      form.reset();
      await loadStudents();
    } catch (error) {
      showNotice(
        "formNotice",
        `${error.message || "Could not save student."} If the browser reports a CORS error, configure API Gateway CORS to allow GET, POST, OPTIONS and the content-type header.`,
        "error"
      );
    } finally {
      saveButton.disabled = false;
      saveButton.textContent = "Save student";
    }
  });

  refreshButton.addEventListener("click", loadStudents);
  form.addEventListener("reset", () => clearNotice("formNotice"));
  loadStudents();
 </script>
</body>
</html>
```

2. create a s3 bucket (mayurcbz.space)
3. Before Uploading Correct index.html file
   change the line - const API_URL = (add your backend URL)
4. Upload the index.html file
5. enable the static website hosting and bucket is public

Copy the static-web-hosting url

http://mayur-cbz-space.com.s3-website.ap-south-1.amazonaws.com

---